Ngày 21 tháng 4 năm 2026

Mục lục

1 Mục tiêu tài liệu **2**

2 Phạm vi và nguồn mã **2**

2.1 Route liên quan 2

2.2 Implementation thuật toán 2

3 Quy ước chung **3**

4 Giải phương trình một ẩn **3**

4.1 Phương pháp chia đôi 4

4.1.1 Bài toán 4

4.1.2 Kiểm tra đầu vào 4

4.1.3 Công thức lặp 4

4.1.4 Sai số và điều kiện dừng 5

4.1.5 Dữ liệu trả về 5

4.1.6 Ghi chú implementation 5

4.2 Phương pháp Newton một ẩn 5

4.2.1 Bài toán 5

4.2.2 Kiểm tra đầu vào 6

4.2.3 Công thức lặp 6

4.2.4 Sai số và điều kiện dừng 6

4.2.5 Dữ liệu trả về 7

4.2.6 Ghi chú implementation 7

4.3 Phương pháp dây cung (Secant) 7

4.3.1 Bài toán 7

4.3.2 Kiểm tra đầu vào 7

4.3.3 Chọn điểm cố định Fourier 7

4.3.4 Công thức lặp 8

4.3.5 Sai số và điều kiện dừng 8

4.3.6 Dữ liệu trả về 8

4.3.7 Ghi chú implementation 9

4.4 Phương pháp lặp đơn cho một ẩn 9

4.4.1 Bài toán . . . 9
4.4.2 Kiểm tra đầu vào . . . 9
4.4.3 Công thức lặp . . . 9
4.4.4 Sai số và điều kiện dừng . . . 9
4.4.5 Dữ liệu trả về . . . 10

5 Giải hệ phương trình tuyến tính **10**
5.1 Khử Gauss . . . 10
5.1.1 Bài toán . . . 10
5.1.2 Ý tưởng thuật toán . . . 10
5.1.3 Chọn pivot . . . 11
5.1.4 Khử . . . 11
5.1.5 Phân loại kết quả . . . 11
5.1.6 Dữ liệu trả về . . . 11
5.1.7 Ghi chú implementation . . . 12
5.2 Gauss–Jordan . . . 12
5.2.1 Bài toán . . . 12
5.2.2 Chọn pivot . . . 12
5.2.3 Chuẩn hóa và khử . . . 12
5.2.4 Phân loại kết quả . . . 12
5.2.5 Dữ liệu trả về . . . 13
5.2.6 Ghi chú implementation . . . 13
5.3 Phân rã LU . . . 13
5.3.1 Bài toán . . . 13
5.3.2 Thuật toán phân rã . . . 13
5.3.3 Giải hệ sau khi phân rã . . . 14
5.3.4 Dữ liệu trả về . . . 14
5.3.5 Ghi chú implementation quan trọng . . . 15
5.4 Cholesky mở rộng . . . 15
5.4.1 Ý tưởng chung . . . 15
5.4.2 Dạng phân rã được dùng . . . 15
5.4.3 Công thức phân rã . . . 15
5.4.4 Nhánh 1: phân rã trực tiếp trên A . . . 16
5.4.5 Nhánh 2: $m < n,\ \mathrm{rank}(A) = m$ . . . 16
5.4.6 Nhánh 3: $\mathrm{rank}(A) = n$ . . . 16
5.4.7 Nhánh 4: fallback khi ma trận làm việc suy biến . . . 17
5.4.8 Dữ liệu trả về . . . 17
5.4.9 Ghi chú implementation quan trọng . . . 18
5.5 Jacobi . . . 18
5.5.1 Bài toán . . . 18
5.5.2 Kiểm tra đầu vào . . . 18
5.5.3 Ma trận lặp . . . 18

5.5.4 Ước lượng sai số . . . 19
5.5.5 Điều kiện dừng . . . 19
5.5.6 Dữ liệu trả về . . . 19
5.6 Gauss–Seidel . . . 20
5.6.1 Bài toán . . . 20
5.6.2 Kiểm tra đầu vào . . . 20
5.6.3 Công thức lặp . . . 20
5.6.4 Hệ số sai số . . . 20
5.6.5 Điều kiện dừng . . . 21
5.6.6 Dữ liệu trả về . . . 21
5.6.7 Ghi chú implementation . . . 21
5.7 Lặp đơn cho hệ tuyến tính . . . 21
5.7.1 Bài toán . . . 21
5.7.2 Kiểm tra đầu vào . . . 21
5.7.3 Chuẩn và điều kiện hội tụ . . . 21
5.7.4 Công thức lặp . . . 22
5.7.5 Điều kiện dừng . . . 22
5.7.6 Dữ liệu trả về . . . 22
5.7.7 Ghi chú implementation . . . 22

6 Giải hệ phương trình phi tuyến 22
6.1 Newton cho hệ phi tuyến . . . 23
6.1.1 Bài toán . . . 23
6.1.2 Tiền xử lý . . . 23
6.1.3 Công thức lặp . . . 23
6.1.4 Kiểm tra trong quá trình lặp . . . 23
6.1.5 Sai số và điều kiện dừng . . . 24
6.1.6 Dữ liệu trả về . . . 24
6.1.7 Ghi chú implementation . . . 24
6.2 Newton cải tiến cho hệ phi tuyến . . . 24
6.2.1 Ý tưởng . . . 24
6.2.2 Kiểm tra đầu vào . . . 25
6.2.3 Sai số và điều kiện dừng . . . 25
6.2.4 Dữ liệu trả về . . . 25
6.2.5 Ghi chú implementation . . . 25
6.3 Lặp đơn cho hệ phi tuyến . . . 25
6.3.1 Bài toán . . . 25
6.3.2 Miền kiểm tra hội tụ . . . 26
6.3.3 Ma trận Jacobi và hệ số co . . . 26
6.3.4 Cách xấp xỉ cực đại trên quả cầu . . . 26
6.3.5 Công thức lặp . . . 27
6.3.6 Điều kiện dừng . . . 27

6.3.7 Dữ liệu trả về 27
6.3.8 Ghi chú implementation quan trọng 28

7 Các điểm đáng review riêng **28**
7.1 LU đang gộp "nghiệm đúng duy nhất" và "nghiệm bình phương tối thiểu duy nhất" 28
7.2 Cholesky hiện tại không phải Cholesky SPD chuẩn 28
7.3 Gauss có pivoting yếu hơn LU 29
7.4 Simple iteration cho hệ phi tuyến đang ghi sai số tương đối nhưng dừng theo ngưỡng tuyệt đối 29
7.5 Newton/Newton hệ phi tuyến đang dùng nghịch đảo tường minh 29
7.6 Các kiểm tra hội tụ theo đạo hàm đều là kiểm tra lấy mẫu 29

8 Mã giả các thuật toán **29**

1 Mục tiêu tài liệu

Tài liệu này mô tả **đúng theo implementation backend hiện tại** của các nhóm thuật toán:

1. Giải phương trình một ẩn.
2. Giải hệ phương trình tuyến tính.
3. Giải hệ phương trình phi tuyến.

Mục tiêu không phải là viết lại lý thuyết thuần túy, mà là ghi rõ:

- bài toán mà từng thuật toán đang xử lý;
- điều kiện đầu vào và các kiểm tra trước khi lặp/khử;
- công thức cốt lõi;
- điều kiện dừng đúng như code hiện tại;
- dữ liệu trả về cho formatter/frontend;
- những điểm implementation khác với cách trình bày chuẩn trong giáo trình.

Lưu ý: trong cụm "giải hệ pt tiếp tuyến", tài liệu này hiểu theo ngữ cảnh codebase là **giải hệ phương trình tuyến tính**. Nếu ý định là "phương pháp tiếp tuyến/Newton", phần đó đã được bao phủ trong mục giải phương trình một ẩn và hệ phi tuyến.

2 Phạm vi và nguồn mã

2.1 Route liên quan

- `backend/routes/root_finding_routes.py`
- `backend/routes/linear_algebra_routes.py`
- `backend/routes/nonlinear_systems_routes.py`

2.2 Implementation thuật toán

- `backend/numerical_methods/root_finding/bisection.py`
- `backend/numerical_methods/root_finding/newton.py`
- `backend/numerical_methods/root_finding/secant.py`
- `backend/numerical_methods/root_finding/simple_iteration.py`
- `backend/numerical_methods/linear_algebra/direct/gauss_elimination.py`

- `backend/numerical_methods/linear_algebra/direct/gauss_jordan.py`
- `backend/numerical_methods/linear_algebra/direct/lu_decomposition.py`
- `backend/numerical_methods/linear_algebra/direct/cholesky.py`
- `backend/numerical_methods/linear_algebra/iterative/jacobi.py`
- `backend/numerical_methods/linear_algebra/iterative/gauss_seidel.py`
- `backend/numerical_methods/linear_algebra/iterative/simple_iteration.py`
- `backend/numerical_methods/nonlinear_systems/newton.py`
- `backend/numerical_methods/nonlinear_systems/newton_modified.py`
- `backend/numerical_methods/nonlinear_systems/simple_iteration.py`

3 Quy ước chung

Các ký hiệu chung:

- `tol`, `tolerance`, `zero_tolerance` là các ngưỡng số học;
- $\operatorname{rank}(A)$ được xác định bằng `np.linalg.matrix_rank(..., tol=...)` ở các nơi có truyền `tol`;
- với hệ tuyến tính, nhiều hàm hỗ trợ cả b là vector cột lẫn ma trận nhiều vế phải.

Tài liệu này dùng các ký hiệu:

$$Ax = b, \qquad F(X) = 0, \qquad X = \Phi(X).$$

Khi nói "nghiệm riêng + không gian null", ta hiểu dưới dạng:

$$x = x_p + Vt,$$

trong đó x_p là một nghiệm riêng, V là ma trận có các cột là cơ sở của $\operatorname{Null}(A)$, và t là vector tham số tự do.

4 Giải phương trình một ẩn

API tương ứng:

- Route: `POST /api/root-finding/solve`
- `method` hỗ trợ: `bisection`, `newton`, `secant`, `simple_iteration`

Các chế độ dừng dùng chung:

- `stop_mode = iterations`: dừng sau số vòng lặp cố định;
- `stop_mode = absolute_error`: dừng theo sai số tuyệt đối;
- `stop_mode = relative_error`: dừng theo sai số tương đối.

Riêng Newton và Secant còn nhận:

- `adv_stop_condition = f_xn`
- `adv_stop_condition = xn_x_prev`

4.1 Phương pháp chia đôi

4.1.1 Bài toán

Tìm nghiệm của

$$f(x) = 0, \qquad x \in [a, b],$$

với điều kiện kỳ vọng là $f(a)$ và $f(b)$ trái dấu.

4.1.2 Kiểm tra đầu vào

Code đang thực hiện:

1. Kiểm tra $f(a)f(b) < 0$.
2. Có một bước kiểm tra đơn điệu gần đúng bằng sai phân trung tâm trên 20 điểm:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \qquad h = 10^{-6}.$$

Tuy nhiên bước này **không chặn thuật toán**. Nếu kết quả không đơn điệu, code vẫn tiếp tục.

4.1.3 Công thức lặp

Ở mỗi bước:

$$c_n = \frac{a_n + b_n}{2}.$$

Sau đó:

- nếu $f(a_n)f(c_n) < 0$ thì cập nhật $b_{n+1} = c_n$;
- ngược lại cập nhật $a_{n+1} = c_n$.

4.1.4 Sai số và điều kiện dừng

Trong code:

$$e_n = |c_n - c_{n-1}|, \qquad e_n^{(\mathrm{rel})} = \frac{|c_n - c_{n-1}|}{|c_n|}.$$

Điều kiện dừng là một trong các trường hợp:

- `mode = iterations`: chạy đúng N bước, trừ khi $f(c_n) = 0$;
- `mode = absolute_error`: dừng khi $e_n < \varepsilon$;
- `mode = relative_error`: dừng khi $e_n^{(\mathrm{rel})} < \varepsilon$;
- ngoài ra nếu $f(c_n) = 0$ thì dừng ngay.

4.1.5 Dữ liệu trả về

- `solution`
- `steps`
- `iterations`

Mỗi phần tử của `steps` có thể chứa:

- `a`, `b`, `c`
- `fc`
- `error`
- `relative_error`

4.1.6 Ghi chú implementation

- Đây là một implementation khá mềm: kiểm tra đơn điệu chỉ mang tính tham khảo.
- Trong chế độ sai số, giá trị `c_prev` ban đầu được gán bằng a, nên sai số ở bước đầu tiên là $|c_1 - a|$.

4.2 Phương pháp Newton một ẩn

4.2.1 Bài toán

Tìm nghiệm của

$$f(x) = 0$$

theo công thức tiếp tuyến:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

4.2.2 Kiểm tra đầu vào

Code kiểm tra gần đúng các điều kiện hội tụ trên đoạn $[a, b]$:

1. $f'(x)$ không đổi dấu trên 20 điểm mẫu;
2. $f''(x)$ không đổi dấu trên 20 điểm mẫu.

Sau đó tính các hằng số trên 500 điểm mẫu:

$$m_1 = \min_{x\in[a,b]} |f'(x)|, \qquad M_2 = \max_{x\in[a,b]} |f''(x)|.$$

Nếu m_1 quá nhỏ, code coi công thức sai số không còn đáng tin và báo lỗi.

4.2.3 Công thức lặp

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Nếu $f'(x_k)$ gần 0, thuật toán dừng với lỗi.
Nếu x_{k+1} ra ngoài đoạn $[a, b]$, thuật toán cũng dừng với lỗi.

4.2.4 Sai số và điều kiện dừng

Nếu chọn `adv_stop_condition = xn_x_prev`, code dùng ước lượng hậu nghiệm:

$$e_k \approx \frac{M_2}{2m_1}(x_{k+1} - x_k)^2.$$

Nếu chọn `adv_stop_condition = f_xn`, code dùng:

$$e_k \approx \frac{|f(x_{k+1})|}{m_1}.$$

Trong chế độ sai số tương đối:

$$e_k^{(\mathrm{rel})} = \frac{e_k}{|x_{k+1}|}.$$

Thuật toán dừng khi:

- `mode = iterations`: đủ số bước yêu cầu;
- `mode = absolute_error`: $e_k < \varepsilon$;
- `mode = relative_error`: $e_k^{(\mathrm{rel})} < \varepsilon$.

4.2.5 Dữ liệu trả về

- `solution`
- `iterations`
- `steps`
- `m1`
- `M2`
- `x0`

4.2.6 Ghi chú implementation

- Điều kiện hội tụ chỉ được kiểm tra bằng lấy mẫu số học, không phải chứng minh giải tích.
- Sai số residual-based của Newton dùng $f(x_{k+1})$, đúng với bước mới nhất.

4.3 Phương pháp dây cung (Secant)

4.3.1 Bài toán

Tìm nghiệm của

$$f(x) = 0$$

bằng dạng dây cung với một điểm cố định Fourier d.

4.3.2 Kiểm tra đầu vào

Code kiểm tra:

1. $f(a)f(b) < 0$;
2. $f'(x)$ không đổi dấu trên 20 điểm mẫu;
3. $f''(x)$ không đổi dấu trên 20 điểm mẫu.

4.3.3 Chọn điểm cố định Fourier

Code chọn:

- nếu $f(a)f''(a) > 0$ thì $d = a,\ x_0 = b$;
- nếu $f(b)f''(b) > 0$ thì $d = b,\ x_0 = a$;
- nếu cả hai không thỏa, code dùng một nhánh nới lỏng dựa trên $x_{\text{mid}} = (a + b)/2$.

Nếu vẫn không chọn được d, thuật toán báo lỗi.

4.3.4 Công thức lặp

Với d là điểm cố định:

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - d)}{f(x_n) - f(d)}.$$

Đồng thời code tính:

$$m_1 = \min_{x\in[a,b]} |f'(x)|, \qquad M_1 = \max_{x\in[a,b]} |f'(x)|.$$

4.3.5 Sai số và điều kiện dừng

Nếu `adv_stop_condition = xn_x_prev`:

$$e_n \approx \frac{M_1 - m_1}{m_1}|x_n - x_{n-1}|.$$

Nếu dùng sai số tương đối:

$$e_n^{(\mathrm{rel})} = \frac{e_n}{|x_n|}.$$

Nếu `adv_stop_condition = f_xn`, code dùng:

$$e_n \approx \frac{|f(x_n)|}{m_1}.$$

Thuật toán dừng khi:

- đủ số bước yêu cầu, hoặc
- sai số tuyệt đối/tương đối nhỏ hơn ngưỡng.

4.3.6 Dữ liệu trả về

- `solution`
- `steps`
- `iterations`
- `m1`
- `M1`
- `d`
- `x0`

4.3.7 Ghi chú implementation

- Khác với Newton, nhánh `f_xn` ở đây dùng $f(x_n)$ của điểm hiện tại trước khi cập nhật, không dùng $f(x_{n+1})$.
- Thuật toán sẽ báo lỗi nếu mẫu số $f(x_n) - f(d)$ tiến về 0.

4.4 Phương pháp lặp đơn cho một ẩn

4.4.1 Bài toán

Giải phương trình điểm bất động:

$$x = \phi(x).$$

4.4.2 Kiểm tra đầu vào

Code quy về:

$$f(x) = \phi(x) - x$$

và kiểm tra:

$$f(a)f(b) < 0.$$

Sau đó ước lượng hệ số co:

$$q = \max_{x \in [a,b]} |\phi'(x)|$$

trên 200 điểm mẫu. Nếu $q \geq 1$, thuật toán báo lỗi.

4.4.3 Công thức lặp

$$x_{k+1} = \phi(x_k).$$

Nếu x_{k+1} ra ngoài đoạn $[a, b]$, thuật toán dừng với lỗi.

4.4.4 Sai số và điều kiện dừng

Code dùng ước lượng:

$$e_k = \frac{q}{1-q}|x_{k+1} - x_k|.$$

Nếu chọn sai số tương đối:

$$e_k^{(\mathrm{rel})} = \frac{e_k}{|x_{k+1}|}.$$

Điều kiện dừng:

- `mode = iterations`: đủ số vòng lặp;
- `mode = absolute_error`: $e_k < \varepsilon$;
- `mode = relative_error`: $e_k^{(\mathrm{rel})} < \varepsilon$.

4.4.5 Dữ liệu trả về

- `solution`
- `iterations`
- `steps`
- `q`
- `x0`

5 Giải hệ phương trình tuyến tính

API tương ứng:

- `POST /api/linear-algebra/solve/gauss`
- `POST /api/linear-algebra/solve/gauss-jordan`
- `POST /api/linear-algebra/solve/lu`
- `POST /api/linear-algebra/solve/cholesky`
- `POST /api/linear-algebra/solve/jacobi`
- `POST /api/linear-algebra/solve/gauss-seidel`
- `POST /api/linear-algebra/solve/simple-iteration`

5.1 Khử Gauss

5.1.1 Bài toán

Giải

$$AX = B.$$

Code hỗ trợ cả:

- A vuông;
- A không vuông;
- B là vector cột hoặc nhiều cột.

5.1.2 Ý tưởng thuật toán

Code ghép ma trận mở rộng:

$$[A \mid B]$$

rồi đưa về dạng bậc thang theo từng cột.

5.1.3 Chọn pivot

Ở cột hiện tại:

- nếu phần tử tại vị trí pivot hiện tại có trị tuyệt đối nhỏ hơn `tol`, code tìm **hàng đầu tiên phía dưới** có phần tử khác 0 theo `tol`;
- nếu tìm được thì đổi hàng;
- nếu không tìm được thì **bỏ qua cột đó**.

Điểm quan trọng: đây **không phải partial pivoting chuẩn theo max-abs**. Nó chỉ tìm hàng khả dụng đầu tiên.

5.1.4 Khử

Với pivot tại (r, c):

$$R_i \leftarrow R_i - \frac{a_{ic}}{a_{rc}} R_r, \qquad i > r.$$

Các phép biến đổi được lưu đầy đủ vào `steps`.

5.1.5 Phân loại kết quả

Sau pha khử:

- nếu xuất hiện hàng
$$[0\ 0\ \dots\ 0 \mid \beta], \qquad \beta \neq 0,$$
thì `status = no_solution`;
- nếu $\operatorname{rank}(A) <$ số ẩn, code dựng:
  - một nghiệm riêng bằng cách cho các ẩn tự do bằng 0,
  - các vector cơ sở của không gian null bằng thế ngược;
- nếu $\operatorname{rank}(A) =$ số ẩn, code giải thế ngược để lấy nghiệm duy nhất.

5.1.6 Dữ liệu trả về

Tùy nhánh, có thể gồm:

- `status`
- `solution`
- `particular_solution`
- `null_space_vectors`
- `steps`

- `backward_steps`
- `rank`
- `num_vars`

5.1.7 Ghi chú implementation

- Vì không dùng max-abs pivoting, độ ổn định số học của Gauss hiện tại thấp hơn LU.
- `rank` thực chất được xác định bằng số pivot cột tìm được trong quá trình khử.

5.2 Gauss–Jordan

5.2.1 Bài toán

Giải

$$AX = B$$

trực tiếp về dạng gần-rút gọn.

5.2.2 Chọn pivot

Thuật toán duyệt trên các hàng và cột chưa được pivot hóa:

1. ưu tiên phần tử đúng bằng 1 hoặc -1;
2. nếu không có, chọn phần tử có trị tuyệt đối lớn nhất trong phần chưa dùng.

Điểm này khác với Gauss thường, vì pivot có thể được chọn ở bất kỳ hàng/cột chưa dùng nào.

5.2.3 Chuẩn hóa và khử

Chuẩn hóa hàng pivot:

$$R_p \leftarrow \frac{1}{a_{pc}} R_p.$$

Sau đó khử mọi hàng còn lại:

$$R_i \leftarrow R_i - a_{ic} R_p, \qquad i \neq p.$$

5.2.4 Phân loại kết quả

- nếu có hàng $0 = \beta$, $\beta \neq 0$ thì vô nghiệm;
- nếu thiếu pivot so với số ẩn thì có vô số nghiệm;
- nếu đủ pivot thì lấy nghiệm trực tiếp từ dạng sau khử.

5.2.5 Dữ liệu trả về

- `status`
- `solution` hoặc `particular_solution`
- `null_space_vectors`
- `steps`
- `rank`
- `num_vars`

5.2.6 Ghi chú implementation

- Sau khi khử, code phải sắp xếp lại cặp (`pivot_col`, `pivot_row`) để dựng nghiệm đúng theo thứ tự biến.
- Đây là một version thiên về hiển thị chi tiết hơn là tối ưu số học.

5.3 Phân rã LU

5.3.1 Bài toán

Giải

$$AX = B$$

bằng phân rã

$$PA = LU.$$

Code hỗ trợ:

- ma trận vuông;
- ma trận chữ nhật;
- ma trận suy biến.

5.3.2 Thuật toán phân rã

Ở cột k, chọn:

$$p = \arg\max_{i \geq k} |u_{ik}|.$$

Nếu pivot đủ lớn:

1. đổi hàng k với p trong U, P;
2. nếu $k > 0$, đổi cả phần tương ứng của L;

3. khử các hàng phía dưới:

$$\ell_{ik} = \frac{u_{ik}}{u_{kk}}, \qquad R_i \leftarrow R_i - \ell_{ik} R_k.$$

Nếu cột không tìm được pivot đủ lớn, code giữ nguyên cột đó và tiếp tục.

5.3.3 Giải hệ sau khi phân rã

Code chia thành các trường hợp:

1. Nếu $\operatorname{rank}(A) < \operatorname{rank}([A \mid B])$:

   `status = no_solution`.

2. Nếu $\operatorname{rank}(A) < n$:
   - `status = infinite_solutions`;
   - một nghiệm riêng được lấy từ `np.linalg.lstsq(A, b, rcond=None)`;
   - không gian null lấy bằng SVD.

3. Nếu $\operatorname{rank}(A) = n$ và A vuông, giải:

$$LY = PB, \qquad UX = Y.$$

4. Nếu $\operatorname{rank}(A) = n$ và A không vuông, code gọi trực tiếp `np.linalg.lstsq(A, b, rcond=None)`.

5.3.4 Dữ liệu trả về

- `status`
- `decomposition = {P, L, U}`
- `lu_steps`
- `forward_steps`
- `backward_steps`
- `solution` hoặc `particular_solution`
- `null_space_vectors`
- `rank`
- `num_vars`

5.3.5 Ghi chú implementation quan trọng

- LU hiện tại hỗ trợ ma trận chữ nhật, điều này không phải lúc nào cũng được trình bày trong giáo trình cơ bản.
- Với trường hợp A không vuông nhưng $\text{rank}(A) = n$, code luôn gán

  `status = unique_solution`,

  trong khi về mặt ý nghĩa toán học, nếu hệ gốc không tương thích thì nghiệm trả về từ `lstsq` là **nghiệm bình phương tối thiểu duy nhất**, không phải nghiệm đúng của $AX = b$.

5.4 Cholesky mở rộng

5.4.1 Ý tưởng chung

Implementation này không giới hạn Cholesky ở dạng SPD truyền thống. Nó triển khai nhiều nhánh:

1. phân rã trực tiếp trên A nếu có thể;
2. chuyển sang hệ chuẩn tắc $A^T A\, X = A^T b$ nếu A đầy cột;
3. chuyển sang hệ đối ngẫu $(AA^T)\Lambda = b$, rồi dựng $X = A^T \Lambda$ nếu A đầy hàng;
4. fallback sang `lstsq + null_space` nếu ma trận làm việc suy biến.

5.4.2 Dạng phân rã được dùng

Code dùng tam giác trên Q:

$$M = Q^T Q$$

chứ không dùng dạng tam giác dưới L như nhiều giáo trình.

5.4.3 Công thức phân rã

Phần tử đường chéo:

$$q_{kk} = \sqrt{m_{kk} - \sum_{s=1}^{k-1} q_{sk}^2}.$$

Phần tử cùng hàng phía bên phải:

$$q_{kj} = \frac{m_{kj} - \sum_{s=1}^{k-1} q_{sk} q_{sj}}{q_{kk}}, \qquad j > k.$$

Sau đó giải:

$$Q^T y = d, \qquad Qz = y.$$

Trong từng nhánh, z là nghiệm trực tiếp hoặc là biến phụ Λ.

5.4.4 Nhánh 1: phân rã trực tiếp trên A

Điều kiện implementation:

- A phải vuông;
- A phải đối xứng theo `np.allclose(A, A.T, atol=tol)`;
- trong quá trình tính q_{kk}, biểu thức dưới căn không được gần 0.

Nếu thỏa, code giải trực tiếp:

$$A = Q^T Q, \qquad Q^T Y = b, \qquad QX = Y.$$

5.4.5 Nhánh 2: $m < n,\ \mathrm{rank}(A) = m$

Đây là hệ thiếu phương trình, đầy hàng. Code chuyển sang hệ đối ngẫu:

$$M = AA^T, \qquad d = b.$$

Giải:

$$M\Lambda = b$$

rồi dựng:

$$X = A^T \Lambda.$$

Kết quả trả về là:

- một nghiệm riêng có chuẩn nhỏ nhất;
- kèm cơ sở của $\mathrm{Null}(A)$ để mô tả họ nghiệm tổng quát.

5.4.6 Nhánh 3: $\mathrm{rank}(A) = n$

Đây là trường hợp ma trận đầy cột. Code lập hệ chuẩn tắc:

$$M = A^T A, \qquad d = A^T b.$$

Sau đó giải:

$$MX = d.$$

Nếu hệ gốc tương thích: `status = unique_solution`.
Nếu hệ gốc không tương thích: `status = least_squares_solution`.

5.4.7 Nhánh 4: fallback khi ma trận làm việc suy biến

Nếu $A^T A$ hoặc AA^T suy biến, code không ném lỗi ngay mà fallback sang:

- `np.linalg.lstsq(A, b, rcond=None)` để lấy một nghiệm riêng;
- SVD để lấy Null(A).

Tùy tình huống, code trả:

- `infinite_solutions`
- `least_squares_infinite`

5.4.8 Dữ liệu trả về

Kết quả của Cholesky là phong phú nhất trong nhóm trực tiếp:

- `status`
- `message`
- `transformation_message`
- `factorization_note`
- `original_factorization`
- `decomposition`
- `solution` hoặc `particular_solution`
- `null_space_vectors`
- `intermediate_y`
- `auxiliary_solution`
- `factorization_steps`
- `forward_steps`
- `backward_steps`
- `reconstruction_steps`
- `residual`
- `residual_norm`
- `rank`
- `num_vars`

5.4.9 Ghi chú implementation quan trọng

- Code làm việc với `dtype=complex`, nên nếu xuất hiện căn bậc hai của số âm, Cholesky **vẫn tiếp tục trong số phức**.
- Vì vậy implementation hiện tại không tương đương hoàn toàn với "Cholesky SPD chuẩn" trong sách.
- Phân rã trực tiếp dùng
$$Q^T Q$$
chứ không dùng liên hợp chuyển vị
$$Q^* Q.$$

5.5 Jacobi

5.5.1 Bài toán

Giải
$$Ax = b$$
bằng lặp Jacobi.

5.5.2 Kiểm tra đầu vào

Code yêu cầu:

- A vuông;
- đường chéo chính không có phần tử gần 0;
- A chéo trội hàng **hoặc** chéo trội cột:
$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$
theo hàng hoặc theo cột.

5.5.3 Ma trận lặp

Code đặt:
$$T = \operatorname{diag}\left(\frac{1}{a_{11}}, \ldots, \frac{1}{a_{nn}}\right),$$
$$B = I - TA, \qquad d = Tb,$$
nên:
$$x^{(k+1)} = Bx^{(k)} + d.$$

5.5.4 Ước lượng sai số

Nếu chéo trội hàng:

- dùng chuẩn ∞;
- hệ số co là $\|B\|_\infty$;
- sai số ước lượng:
$$e_k \approx \frac{\|B\|_\infty}{1 - \|B\|_\infty} \|x^{(k+1)} - x^{(k)}\|_\infty.$$

Nếu chéo trội cột:

- dùng chuẩn 1;
- code tính thêm
$$\lambda = \frac{\max_i |a_{ii}|}{\min_i |a_{ii}|}$$
và
$$e_k \approx \lambda \cdot \frac{\|I - AT\|_1}{1 - \|I - AT\|_1} \|x^{(k+1)} - x^{(k)}\|_1.$$

5.5.5 Điều kiện dừng

Thuật toán dừng khi:
$$e_k < \varepsilon.$$
Nếu không sẽ báo không hội tụ sau `max_iter`.

5.5.6 Dữ liệu trả về

- `solution`
- `iterations`
- `iterations_data`
- `contraction_coefficient`
- `norm_used`
- `is_row_dominant`
- `is_col_dominant`
- `matrix_B`
- `vector_d`

5.6 Gauss–Seidel

5.6.1 Bài toán

Giải

$$Ax = b$$

bằng Gauss–Seidel.

5.6.2 Kiểm tra đầu vào

Code yêu cầu:

- A vuông;
- đường chéo chính không có phần tử gần 0;
- A chéo trội hàng hoặc chéo trội cột.

5.6.3 Công thức lặp

Mỗi thành phần được cập nhật theo dạng:

$$x_j^{(k+1)} = \frac{1}{a_{jj}} \left(b_j - \sum_{i<j} a_{ji} x_i^{(k+1)} - \sum_{i>j} a_{ji} x_i^{(k)} \right).$$

Trong code, vế phải được tính đúng theo nguyên tắc "dùng giá trị mới ở vế trái, giá trị cũ ở vế phải".

5.6.4 Hệ số sai số

Nếu chéo trội hàng, code đặt:

$$s = 0, \qquad q = \max_i \frac{\sum_{j<i} |a_{ij}|}{|a_{ii}| - \sum_{j>i} |a_{ij}|}.$$

Nếu chéo trội cột, code dùng:

$$s = \max_j \frac{\sum_{i>j} |a_{ij}|}{|a_{jj}|}, \qquad q = \max_j \frac{\sum_{i<j} |a_{ij}|}{|a_{jj}| - \sum_{i>j} |a_{ij}|}.$$

Sai số được ước lượng bởi:

$$e_k \approx \frac{q}{(1-s)(1-q)} \|x^{(k+1)} - x^{(k)}\|,$$

theo chuẩn ∞ hoặc 1 tương ứng.

5.6.5 Điều kiện dừng

Thuật toán dừng khi:

$$e_k < \varepsilon.$$

5.6.6 Dữ liệu trả về

- `solution`
- `iterations`
- `iterations_data`
- `coeff_q`
- `coeff_s`
- `norm_used`
- `is_row_dominant`

5.6.7 Ghi chú implementation

- Kết quả trả về hiện có `is_row_dominant`, nhưng không trả trực tiếp `is_col_dominant` như Jacobi.

5.7 Lặp đơn cho hệ tuyến tính

5.7.1 Bài toán

Giải hệ ở dạng:

$$x = Bx + d.$$

5.7.2 Kiểm tra đầu vào

Code yêu cầu:

- B vuông;
- kích thước của d, x_0 phù hợp với B.

5.7.3 Chuẩn và điều kiện hội tụ

Frontend truyền `norm_choice = inf` hoặc `1`.

Code tính:

$$\|B\|$$

theo chuẩn tương ứng.

Nếu

$$\|B\| \geq 1,$$

code **không chặn thuật toán**, chỉ sinh `warning_message`.

5.7.4 Công thức lặp

$$x^{(k+1)} = Bx^{(k)} + d.$$

5.7.5 Điều kiện dừng

Nếu $\|B\| < 1$, code dùng ngưỡng nội bộ:

$$T_{\text{stop}} = \left| \frac{1 - \|B\|}{\|B\|} \right| \varepsilon.$$

trong đó T_{stop} tương ứng với biến `stopping_threshold`.

Nếu $\|B\| \geq 1$, code dùng trực tiếp:

$$T_{\text{stop}} = \varepsilon.$$

Sau đó dừng khi:

$$\|x^{(k+1)} - x^{(k)}\| < T_{\text{stop}}.$$

5.7.6 Dữ liệu trả về

- `solution`
- `iterations`
- `iterations_data`
- `norm_B`
- `warning_message`
- `norm_used`
- `stopping_threshold`

5.7.7 Ghi chú implementation

- Code không nhân thêm hệ số hậu nghiệm kiểu $\|B\|/(1 - \|B\|)$ vào sai số báo cáo; thay vào đó nó biến đổi trực tiếp ngưỡng dừng.

6 Giải hệ phương trình phi tuyến

API tương ứng:

- Route: `POST /api/nonlinear-systems/solve`

- `method` hỗ trợ:
  - `newton`
  - `newton_modified`
  - `simple_iteration`

Điều kiện dừng:

- `stop_option = iterations`
- `stop_option = absolute_error`
- `stop_option = relative_error`

6.1 Newton cho hệ phi tuyến

6.1.1 Bài toán

Giải hệ:

$$F(X) = 0, \qquad F = (f_1, \ldots, f_n)^T.$$

6.1.2 Tiền xử lý

Code:

1. parse danh sách biểu thức thành vector symbolic F;
2. tạo ma trận Jacobi:
$$J(X) = \frac{\partial F}{\partial X};$$
3. dựng thêm các bản display latex cho frontend.

6.1.3 Công thức lặp

Ở mỗi bước:

$$\Delta X_k = J(X_k)^{-1} F(X_k), \qquad X_{k+1} = X_k - \Delta X_k.$$

6.1.4 Kiểm tra trong quá trình lặp

Tại mỗi bước:

- code tính $J(X_k)$;
- nếu $|\det J(X_k)| < 10^{-12}$, thuật toán báo Jacobi suy biến và dừng.

6.1.5 Sai số và điều kiện dừng

Nếu `stop_option = iterations` thì chạy đúng số vòng lặp được yêu cầu.

Nếu dừng theo sai số, code dùng:

$$\Delta_k = X_{k+1} - X_k.$$

Nếu `norm_choice = 1`:

$$e_k = \|\Delta_k\|_1, \qquad e_k^{(\mathrm{rel})} = \frac{\|\Delta_k\|_1}{\|X_{k+1}\|_1}.$$

Ngược lại, mặc định dùng chuẩn vô cùng:

$$e_k = \|\Delta_k\|_\infty, \qquad e_k^{(\mathrm{rel})} = \frac{\|\Delta_k\|_\infty}{\|X_{k+1}\|_\infty}.$$

Điều kiện dừng:

- `absolute_error`: $e_k < \varepsilon$;
- `relative_error`: $e_k^{(\mathrm{rel})} < \varepsilon$.

6.1.6 Dữ liệu trả về

- `status`
- `solution`
- `iterations`
- `jacobian_matrix_latex`
- `steps`
- `message`

6.1.7 Ghi chú implementation

- Code nghịch đảo Jacobi một cách tường minh ở từng bước (`J_val.inv()`), không giải hệ tuyến tính phụ.
- Kiểm tra suy biến bằng định thức là trực quan, nhưng có thể nhạy cảm số học.

6.2 Newton cải tiến cho hệ phi tuyến

6.2.1 Ý tưởng

Khác với Newton chuẩn, Jacobi chỉ được tính và nghịch đảo **một lần tại** X_0:

$$J_0 = J(X_0), \qquad X_{k+1} = X_k - J_0^{-1} F(X_k).$$

6.2.2 Kiểm tra đầu vào

Nếu

$$|\det J(X_0)| < 10^{-12},$$

thì thuật toán dừng ngay với lỗi.

6.2.3 Sai số và điều kiện dừng

Giống Newton chuẩn:

- sai số tuyệt đối/tương đối được tính từ $X_{k+1} - X_k$;
- chuẩn là 1 hoặc ∞.

6.2.4 Dữ liệu trả về

- `status`
- `solution`
- `iterations`
- `J0_inv_matrix`
- `steps`
- `message`

6.2.5 Ghi chú implementation

Đây là một trade-off rõ ràng:

- ít tốn chi phí tính Jacobi/nghịch đảo;
- nhưng độ hội tụ có thể chậm hơn hoặc mất hội tụ nếu $J(X)$ thay đổi mạnh.

6.3 Lặp đơn cho hệ phi tuyến

6.3.1 Bài toán

Giải hệ điểm bất động:

$$X = \Phi(X),$$

với:

- X_0 là điểm đầu;
- α là tâm quả cầu kiểm tra hội tụ;
- r là bán kính quả cầu.

6.3.2 Miền kiểm tra hội tụ

Code làm việc trên quả cầu:

$$\overline{B}(\alpha, r) = \{X : \|X - \alpha\| \leq r\}$$

và yêu cầu $r > 0$.

6.3.3 Ma trận Jacobi và hệ số co

Tính:

$$J_\Phi(X) = \frac{\partial \Phi}{\partial X}.$$

Sau đó lập hai loại biểu thức:

1. Tổng hàng:

$$R_i(X) = \sum_{j=1}^{n} \left| \frac{\partial \phi_i}{\partial x_j}(X) \right|;$$

2. Tổng cột:

$$C_j(X) = \sum_{i=1}^{n} \left| \frac{\partial \phi_i}{\partial x_j}(X) \right|.$$

Code xấp xỉ:

$$K_\infty = \max_i \sup_{X \in \overline{B}(\alpha,r)} R_i(X), \qquad K_1 = \max_j \sup_{X \in \overline{B}(\alpha,r)} C_j(X).$$

Rồi chọn:

$$K = \min(K_\infty, K_1)$$

và lấy chuẩn tương ứng với giá trị nhỏ hơn.

Nếu $K \geq 1$, thuật toán báo không thỏa điều kiện hội tụ.

6.3.4 Cách xấp xỉ cực đại trên quả cầu

Code không giải bài toán tối ưu chính xác, mà dùng lấy mẫu xấp xỉ:

- điểm tâm;
- các điểm trục $\alpha \pm re_i$;
- lưới 5 điểm mỗi chiều nếu dim ≤ 3;
- các đỉnh kiểu $(\pm 1, \ldots, \pm 1)$ đã scale nếu $4 \leq$ dim ≤ 10;
- mẫu ngẫu nhiên trong lòng và trên biên quả cầu với seed cố định 0.

Do đó K_∞ và K_1 là **ước lượng số học**, không phải giá trị giải tích chính xác.

6.3.5 Công thức lặp

$$X_{k+1} = \Phi(X_k).$$

6.3.6 Điều kiện dừng

Nếu `stop_option = iterations` thì chạy đúng N bước.

Nếu dừng theo sai số, code dùng:

$$\Delta_k = X_{k+1} - X_k$$

và lấy

$$\|\Delta_k\|_*$$

theo chuẩn đã chọn tự động từ K_∞ hoặc K_1.

Sau đó lập ngưỡng tiên nghiệm:

$$\varepsilon_{\text{prior}} = \begin{cases} \varepsilon \dfrac{1-K}{K}, & K > 10^{-12}, \\ \varepsilon, & \text{ngược lại.} \end{cases}$$

trong đó $\varepsilon_{\text{prior}}$ tương ứng với biến `priori_tol`.

Thuật toán dừng khi:

$$\|\Delta_k\|_* < \varepsilon_{\text{prior}}.$$

Nếu người dùng chọn `relative_error`, code vẫn ghi thêm:

$$e_k^{(\text{rel})} = \frac{\|\Delta_k\|_*}{\|X_{k+1}\|_*},$$

nhưng **điều kiện dừng thực tế vẫn là bất đẳng thức tuyệt đối ở trên**.

6.3.7 Dữ liệu trả về

- `status`
- `solution`
- `iterations`
- `steps`
- `jacobian_matrix_latex`
- `sphere_center`
- `sphere_radius`
- `row_sum_expressions_latex`

- `row_sum_maxima`
- `col_sum_expressions_latex`
- `col_sum_maxima`
- `contraction_factor_infinity`
- `contraction_factor_one`
- `contraction_factor_K`
- `norm_used_for_K`

6.3.8 Ghi chú implementation quan trọng

- Đây là solver phi tuyến có logic kiểm tra hội tụ chặt nhất, nhưng cũng là solver phụ thuộc nhiều nhất vào xấp xỉ số học.
- Trong nhánh lỗi, hàm này không `raise` như Newton, mà trả trực tiếp

  $$\{\texttt{status: error, ...}\}$$

  để route/formatter xử lý tiếp.

7 Các điểm đáng review riêng

7.1 LU đang gộp "nghiệm đúng duy nhất" và "nghiệm bình phương tối thiểu duy nhất"

Ở nhánh A không vuông nhưng $\text{rank}(A) = n$, code dùng `np.linalg.lstsq` nhưng vẫn trả: `status = unique_solution`. Điều này đúng về mặt "vector trả về là duy nhất", nhưng chưa đúng về semantics nếu hệ gốc vô nghiệm.

7.2 Cholesky hiện tại không phải Cholesky SPD chuẩn

Implementation cho phép:

- tiếp tục trong số phức;
- làm việc với Q^TQ;
- dùng trên hệ chuẩn tắc và hệ đối ngẫu;
- fallback sang `lstsq`.

Vì vậy cần xem đây là một **solver kiểu Cholesky mở rộng**, không phải bản textbook thuần túy.

7.3 Gauss có pivoting yếu hơn LU

Gauss chỉ đổi với hàng khả dụng đầu tiên khi pivot hiện tại gần 0, còn LU dùng max-abs partial pivoting.

7.4 Simple iteration cho hệ phi tuyến đang ghi sai số tương đối nhưng dừng theo ngưỡng tuyệt đối

Ở nhánh `relative_error`, code vẫn break theo:

$$\|\Delta_k\|_* < \varepsilon_{\text{prior}}$$

chứ không phải theo sai số tương đối đã hiển thị.

7.5 Newton/Newton hệ phi tuyến đang dùng nghịch đảo tường minh

Việc dùng

$$J^{-1}F$$

hoặc $\det(J)$ để kiểm tra suy biến là dễ hiểu và tiện hiển thị, nhưng không phải cách ổn định số học nhất nếu sau này muốn scale lên hệ lớn hơn.

7.6 Các kiểm tra hội tụ theo đạo hàm đều là kiểm tra lấy mẫu

Trong các solver Newton/Secant/Lặp đơn 1 ẩn, điều kiện hội tụ được kiểm tra trên tập điểm mẫu rời rạc. Điều này phù hợp cho ứng dụng học tập và UI, nhưng không nên diễn đạt như một chứng minh toán học chặt.

8 Mã giả các thuật toán

Algorithm 1 Phương pháp chia đôi

Input:

- Hàm số $f(x)$.
- Khoảng cách ly nghiệm $[a, b]$ sao cho $f(a)f(b) < 0$.
- Chế độ dừng `stop_mode`: `iterations`, `absolute_error`, hoặc `relative_error`.
- Ngưỡng sai số ε hoặc số vòng lặp tối đa N.

Output:

- Nghiệm gần đúng x^*.
- Số lần lặp.
- Bảng các bước lặp: $a_n, b_n, c_n, f(c_n)$, sai số tuyệt đối và sai số tương đối.

Thuật toán:

B1. Kiểm tra điều kiện đầu vào: nếu $f(a)f(b) \geq 0$ thì dừng và báo lỗi.

B2. Khởi tạo $n = 0$, $c_{\text{prev}} = a$.

B3. Lặp:

- Tính $c_n = \dfrac{a_n + b_n}{2}$.
- Tính $f(c_n)$.
- Tính sai số tuyệt đối $e_n = |c_n - c_{\text{prev}}|$.
- Nếu $c_n \neq 0$, tính sai số tương đối $e_n^{(\text{rel})} = \dfrac{|c_n - c_{\text{prev}}|}{|c_n|}$.
- Ghi lại bước lặp.
- Nếu $f(c_n) = 0$, chuyển tới B4.
- Kiểm tra điều kiện dừng:
  - nếu `stop_mode = iterations` và $n = N$, chuyển tới B4;
  - nếu `stop_mode = absolute_error` và $e_n < \varepsilon$, chuyển tới B4;
  - nếu `stop_mode = relative_error` và $e_n^{(\text{rel})} < \varepsilon$, chuyển tới B4.
- Cập nhật khoảng:
  - nếu $f(a_n)f(c_n) < 0$ thì đặt $b_{n+1} = c_n$, $a_{n+1} = a_n$;
  - ngược lại đặt $a_{n+1} = c_n$, $b_{n+1} = b_n$.
- Đặt $c_{\text{prev}} = c_n$, $n = n + 1$.

B4. Trả ra nghiệm gần đúng $x^* = c_n$.

Algorithm 2 Phương pháp Newton một ẩn

Input:

- Hàm số $f(x)$, đạo hàm $f'(x)$, đạo hàm bậc hai $f''(x)$.
- Khoảng khảo sát $[a, b]$.
- Điểm đầu x_0.

- Chế độ dừng `stop_mode`.
- Tiêu chuẩn dừng nâng cao `adv_stop_condition`: `xn_x_prev` hoặc `f_xn`.
- Sai số ε hoặc số bước N.

Output:

- Nghiệm gần đúng x^*.
- Số bước lặp.
- Các giá trị m_1, M_2.
- Nhật ký các bước lặp.

Thuật toán:

B1. Kiểm tra gần đúng điều kiện hội tụ trên $[a, b]$: $f'(x)$ không đổi dấu và $f''(x)$ không đổi dấu.

B2. Tính $m_1 = \min_{x\in[a,b]} |f'(x)|$, $M_2 = \max_{x\in[a,b]} |f''(x)|$. Nếu m_1 quá nhỏ thì dừng và báo lỗi.

B3. Khởi tạo $n = 0$, $x_n = x_0$.

B4. Lặp:

- Nếu $f'(x_n) \approx 0$ thì dừng và báo lỗi.
- Tính $x_{n+1} = x_n - \dfrac{f(x_n)}{f'(x_n)}$.
- Nếu $x_{n+1} \notin [a, b]$ thì dừng và báo lỗi.
- Tính sai số:
  - nếu `adv_stop_condition = xn_x_prev` thì $e_n \approx \dfrac{M_2}{2m_1}(x_{n+1} - x_n)^2$;
  - nếu `adv_stop_condition = f_xn` thì $e_n \approx \dfrac{|f(x_{n+1})|}{m_1}$.
- Nếu $x_{n+1} \neq 0$, tính $e_n^{(\text{rel})} = \dfrac{e_n}{|x_{n+1}|}$.
- Ghi lại bước lặp.
- Kiểm tra điều kiện dừng:
  - nếu `stop_mode = iterations` và $n + 1 = N$, chuyển tới B5;
  - nếu `stop_mode = absolute_error` và $e_n < \varepsilon$, chuyển tới B5;
  - nếu `stop_mode = relative_error` và $e_n^{(\text{rel})} < \varepsilon$, chuyển tới B5.
- Cập nhật $x_n = x_{n+1}$, $n = n + 1$.

B5. Trả ra nghiệm gần đúng $x^* = x_{n+1}$.

Algorithm 3 **Phương pháp dây cung với điểm cố định Fourier**

Input:

- Hàm số $f(x)$, $f'(x)$, $f''(x)$.
- Khoảng $[a, b]$ sao cho $f(a)f(b) < 0$.
- Chế độ dừng `stop_mode`.
- Tiêu chuẩn dừng `adv_stop_condition`.
- Sai số ε hoặc số bước N.

Output:

- Nghiệm gần đúng x^*.
- Điểm cố định Fourier d.
- Điểm bắt đầu x_0.
- Các giá trị m_1, M_1.
- Nhật ký các bước lặp.

Thuật toán:

B1. Kiểm tra $f(a)f(b) < 0$. Nếu không thỏa thì dừng.

B2. Kiểm tra gần đúng điều kiện hội tụ: $f'(x)$, $f''(x)$ không đổi dấu trên $[a, b]$.

B3. Chọn điểm cố định Fourier:

- nếu $f(a)f''(a) > 0$ thì đặt $d = a$, $x_0 = b$;
- nếu $f(b)f''(b) > 0$ thì đặt $d = b$, $x_0 = a$;
- nếu không chọn được thì dùng nhánh nới lỏng dựa trên trung điểm.

Nếu vẫn không chọn được d, dừng và báo lỗi.

B4. Tính $m_1 = \min_{x\in[a,b]} |f'(x)|$, $M_1 = \max_{x\in[a,b]} |f'(x)|$.

B5. Khởi tạo $n = 0$, $x_n = x_0$.

B6. Lặp:

- Nếu $f(x_n) - f(d) \approx 0$ thì dừng và báo lỗi.

- Tính $x_{n+1} = x_n - \dfrac{f(x_n)(x_n - d)}{f(x_n) - f(d)}$.
- Tính sai số:
  - nếu `adv_stop_condition = xn_x_prev` thì $e_n \approx \dfrac{M_1 - m_1}{m_1}|x_n - x_{n-1}|$;
  - nếu `adv_stop_condition = f_xn` thì $e_n \approx \dfrac{|f(x_n)|}{m_1}$.
- Nếu $x_n \neq 0$, tính $e_n^{(\text{rel})} = \dfrac{e_n}{|x_n|}$.
- Ghi lại bước lặp.
- Kiểm tra điều kiện dừng: đủ số vòng lặp, hoặc $e_n < \varepsilon$, hoặc $e_n^{(\text{rel})} < \varepsilon$.
- Cập nhật $x_n = x_{n+1}$, $n = n + 1$.

B7. Trả ra nghiệm gần đúng $x^* = x_{n+1}$.

Algorithm 4 Phương pháp lặp đơn cho một ẩn

Input:

- Hàm lặp $\phi(x)$.
- Khoảng $[a, b]$.
- Giá trị đầu x_0.
- Chế độ dừng `stop_mode`.
- Sai số ε hoặc số vòng lặp N.

Output:

- Nghiệm gần đúng x^*.
- Hệ số co q.
- Bảng các bước lặp.

Thuật toán:

B1. Lập $f(x) = \phi(x) - x$ và kiểm tra $f(a)f(b) < 0$.

B2. Ước lượng $q = \max_{x\in[a,b]} |\phi'(x)|$. Nếu $q \geq 1$, dừng và báo lỗi.

B3. Khởi tạo $n = 0$, $x_n = x_0$.

B4. Lặp:

- Tính $x_{n+1} = \phi(x_n)$.
- Nếu $x_{n+1} \notin [a, b]$, dừng và báo lỗi.
- Tính sai số hậu nghiệm $e_n = \dfrac{q}{1-q}|x_{n+1} - x_n|$.
- Nếu $x_{n+1} \neq 0$, tính $e_n^{(\text{rel})} = \dfrac{e_n}{|x_{n+1}|}$.
- Ghi lại bước lặp.
- Kiểm tra điều kiện dừng: theo số vòng lặp, hoặc $e_n < \varepsilon$, hoặc $e_n^{(\text{rel})} < \varepsilon$.
- Cập nhật $x_n = x_{n+1}$, $n = n + 1$.

B5. Trả ra nghiệm gần đúng $x^* = x_{n+1}$.

Algorithm 5 Khử Gauss

Input:

- Ma trận hệ số $A \in \mathbb{R}^{m \times n}$.
- Vế phải B.
- Ngưỡng `tol`.

Output:

- Trạng thái hệ: nghiệm duy nhất, vô nghiệm, hoặc vô số nghiệm.
- Nếu có nghiệm duy nhất: nghiệm X.
- Nếu vô số nghiệm: nghiệm riêng và các vector cơ sở của $\text{Null}(A)$.
- Các bước khử và thế ngược.

Thuật toán:

B1. Lập ma trận mở rộng $M = [A \mid B]$.

B2. Khởi tạo hàng pivot hiện tại $r = 1$.

B3. Với mỗi cột $c = 1, 2, \ldots, n$:

- Nếu $r > m$, dừng quá trình khử.
- Nếu $|M_{rc}| <$ `tol`, tìm hàng đầu tiên $i > r$ sao cho $|M_{ic}| \geq$ `tol`.
- Nếu tìm được thì đổi hàng $R_r \leftrightarrow R_i$.
- Nếu không tìm được thì bỏ qua cột c và chuyển sang cột kế tiếp.

- Với mỗi hàng $i > r$, tính $\lambda = \dfrac{M_{ic}}{M_{rc}}$, rồi cập nhật $R_i \leftarrow R_i - \lambda R_r$.
- Ghi lại bước khử.
- Tăng chỉ số pivot: $r = r + 1$.

B4. Kiểm tra hàng mâu thuẫn $[0\ 0\ \ldots\ 0 \mid \beta]$, $\beta \neq 0$. Nếu có, kết luận hệ vô nghiệm.

B5. Xác định hạng của A và số ẩn.

B6. Nếu $\text{rank}(A) = n$, thực hiện thế ngược để tìm nghiệm duy nhất.

B7. Nếu $\text{rank}(A) < n$, chọn các ẩn tự do bằng 0, tìm một nghiệm riêng, rồi dựng các vector cơ sở của $\text{Null}(A)$.

B8. Trả ra kết quả.

Algorithm 6 Gauss–Jordan

Input:

- Ma trận A.
- Vế phải B.
- Ngưỡng `tol`.

Output:

- Trạng thái hệ.
- Nghiệm hoặc nghiệm riêng và không gian null.
- Các bước pivot, chuẩn hóa, khử.

Thuật toán:

B1. Lập ma trận mở rộng $M = [A \mid B]$.

B2. Khởi tạo tập hàng và cột chưa dùng.

B3. Lặp cho đến khi không chọn được pivot mới:

- Tìm pivot theo ưu tiên: ưu tiên phần tử bằng 1 hoặc -1; nếu không có thì chọn phần tử có trị tuyệt đối lớn nhất trong phần chưa dùng.
- Nếu không tìm được pivot đủ lớn thì dừng.
- Chuẩn hóa hàng pivot: $R_p \leftarrow \dfrac{1}{M_{pc}} R_p$.

- Với mọi hàng $i \neq p$, khử: $R_i \leftarrow R_i - M_{ic}R_p$.
- Đánh dấu hàng và cột pivot đã dùng.
- Ghi lại bước khử.

B4. Nếu có hàng mâu thuẫn $0 = \beta$, $\beta \neq 0$, kết luận vô nghiệm.

B5. Nếu số pivot bằng số ẩn, đọc trực tiếp nghiệm từ ma trận sau khử.

B6. Nếu số pivot nhỏ hơn số ẩn, kết luận vô số nghiệm; dựng nghiệm riêng và không gian null.

B7. Trả ra kết quả.

Algorithm 7 Phân rã LU

Input:

- Ma trận A.
- Vế phải B.
- Ngưỡng `tol`.

Output:

- Các ma trận P, L, U.
- Trạng thái hệ.
- Nghiệm, hoặc nghiệm riêng và không gian null.
- Các bước phân rã, thế xuôi, thế ngược.

Thuật toán:

B1. Khởi tạo $P = I$, $L = I$, $U = A$.

B2. Với mỗi cột k:

- Tìm chỉ số pivot $p = \arg\max_{i \geq k} |u_{ik}|$.
- Nếu $|u_{pk}| <$ `tol`, bỏ qua cột này.
- Nếu $p \neq k$, đổi hàng trong U, P, và phần thích hợp của L.
- Với mỗi hàng $i > k$: $\ell_{ik} = \dfrac{u_{ik}}{u_{kk}}$, rồi cập nhật $R_i \leftarrow R_i - \ell_{ik}R_k$.
- Ghi lại bước phân rã.

B3. Xét hạng của A và ma trận mở rộng $[A \mid B]$:

- nếu $\text{rank}(A) < \text{rank}([A \mid B])$ thì vô nghiệm;
- nếu $\text{rank}(A) < n$ thì vô số nghiệm;
- nếu $\text{rank}(A) = n$ và A vuông thì giải thế xuôi, thế ngược;
- nếu $\text{rank}(A) = n$ và A không vuông thì dùng nghiệm bình phương tối thiểu.

B4. Nếu A vuông và đầy hạng, giải $LY = PB$ rồi $UX = Y$.

B5. Trả ra P, L, U và nghiệm.

Algorithm 8 Cholesky mở rộng

Input:

- Ma trận A.
- Vế phải b.
- Ngưỡng `tol`.

Output:

- Ma trận phân rã Q nếu có.
- Trạng thái hệ.
- Nghiệm hoặc nghiệm bình phương tối thiểu.
- Các bước phân rã và giải hệ phụ.

Thuật toán:

B1. Nếu A vuông và đối xứng, thử phân rã trực tiếp $A = Q^T Q$.

B2. Với từng k, tính $q_{kk} = \sqrt{a_{kk} - \sum_{s=1}^{k-1} q_{sk}^2}$, và với $j > k$, tính $q_{kj} = \dfrac{a_{kj} - \sum_{s=1}^{k-1} q_{sk} q_{sj}}{q_{kk}}$.

B3. Nếu phân rã trực tiếp thành công, giải $Q^T y = b$ rồi $Qx = y$.

B4. Nếu A đầy cột, lập hệ chuẩn tắc $M = A^T A$, $d = A^T b$, rồi giải $MX = d$.

B5. Nếu A đầy hàng, lập hệ đối ngẫu $M = AA^T$, $d = b$, giải $M\Lambda = b$, rồi dựng $X = A^T \Lambda$.

B6. Nếu ma trận làm việc suy biến, chuyển sang nghiệm bình phương tối thiểu và dựng không gian null bằng SVD.

B7. Trả ra kết quả cuối cùng.

Algorithm 9 Jacobi

Input:

- Ma trận vuông A.
- Vector b.
- Giá trị đầu $x^{(0)}$.
- Sai số ε.
- Số vòng lặp tối đa `max_iter`.

Output:

- Nghiệm gần đúng x^*.
- Số vòng lặp.
- Ma trận lặp B, vector d.
- Bảng dữ liệu các bước lặp.

Thuật toán:

B1. Kiểm tra: A vuông, các phần tử đường chéo chính khác 0, và A chéo trội hàng hoặc chéo trội cột.

B2. Lập $T = \text{diag}(1/a_{11}, \ldots, 1/a_{nn})$, $B = I - TA$, $d = Tb$.

B3. Khởi tạo $k = 0$, $x^{(k)} = x^{(0)}$.

B4. Lặp:

- Tính $x^{(k+1)} = Bx^{(k)} + d$.
- Tính sai số hậu nghiệm theo chuẩn phù hợp.
- Nếu $e_k < \varepsilon$, chuyển tới B5.
- Ghi lại bước lặp.
- Cập nhật $k = k + 1$.
- Nếu $k =$ `max_iter`, dừng và báo không hội tụ.

B5. Trả ra nghiệm gần đúng $x^* = x^{(k+1)}$.

Algorithm 10 **Gauss–Seidel**

Input:

- Ma trận A.
- Vector b.
- Giá trị đầu $x^{(0)}$.
- Sai số ε.
- Số vòng lặp tối đa.

Output:

- Nghiệm gần đúng x^*.
- Các hệ số q, s.
- Nhật ký lặp.

Thuật toán:

B1. Kiểm tra A vuông, đường chéo chính khác 0, và điều kiện chéo trội.

B2. Khởi tạo $k = 0$, $x^{(k)} = x^{(0)}$.

B3. Lặp:

- Với mỗi $j = 1, \ldots, n$, tính

$$x_j^{(k+1)} = \frac{1}{a_{jj}} \left(b_j - \sum_{i<j} a_{ji} x_i^{(k+1)} - \sum_{i>j} a_{ji} x_i^{(k)} \right).$$

- Tính sai số hậu nghiệm $e_k \approx \dfrac{q}{(1-s)(1-q)} \|x^{(k+1)} - x^{(k)}\|$.
- Ghi lại bước lặp.
- Nếu $e_k < \varepsilon$, chuyển tới B4.
- Cập nhật $x^{(k)} = x^{(k+1)}$, $k = k + 1$.

B4. Trả ra nghiệm gần đúng $x^* = x^{(k+1)}$.

Algorithm 11 **Lặp đơn cho hệ tuyến tính**

Input:

- Ma trận B.
- Vector d.
- Giá trị đầu $x^{(0)}$.
- Chuẩn chọn: 1 hoặc ∞.
- Sai số ε.
- Số vòng lặp tối đa.

Output:

- Nghiệm gần đúng x^*.
- Giá trị $\|B\|$.
- Cảnh báo hội tụ nếu có.
- Dữ liệu các bước lặp.

Thuật toán:

B1. Kiểm tra B vuông và kích thước của d, $x^{(0)}$ phù hợp.

B2. Tính $q = \|B\|$.

B3. Nếu $q \geq 1$, sinh cảnh báo nhưng vẫn tiếp tục.

B4. Xác định ngưỡng dừng

$$T_{\text{stop}} = \begin{cases} \left|\dfrac{1-q}{q}\right|\varepsilon, & q < 1, \\ \varepsilon, & q \geq 1. \end{cases}$$

B5. Khởi tạo $k = 0$, $x^{(k)} = x^{(0)}$.

B6. Lặp:

- Tính $x^{(k+1)} = Bx^{(k)} + d$.
- Nếu $\|x^{(k+1)} - x^{(k)}\| < T_{\text{stop}}$, chuyển tới B7.
- Ghi lại bước lặp.

- Cập nhật $x^{(k)} = x^{(k+1)}$, $k = k + 1$.

B7. Trả ra nghiệm gần đúng $x^* = x^{(k+1)}$.

Algorithm 12 Newton cho hệ phi tuyến

Input:

- Hệ $F(X) = 0$.
- Điểm đầu X_0.
- Chuẩn chọn: 1 hoặc ∞.
- Chế độ dừng `stop_option`.
- Sai số ε hoặc số vòng lặp N.

Output:

- Nghiệm gần đúng X^*.
- Ma trận Jacobi.
- Nhật ký các bước lặp.

Thuật toán:

B1. Lập vector hàm $F(X) = (f_1, \ldots, f_n)^T$.

B2. Tính ma trận Jacobi $J(X) = \dfrac{\partial F}{\partial X}$.

B3. Khởi tạo $k = 0$, $X_k = X_0$.

B4. Lặp:

- Tính $J(X_k)$. Nếu $|\det J(X_k)| < 10^{-12}$, dừng và báo lỗi.
- Tính $\Delta X_k = J(X_k)^{-1} F(X_k)$.
- Cập nhật $X_{k+1} = X_k - \Delta X_k$.
- Tính sai số $e_k = \|X_{k+1} - X_k\|$, $e_k^{(\text{rel})} = \dfrac{\|X_{k+1} - X_k\|}{\|X_{k+1}\|}$.
- Ghi lại bước lặp.
- Kiểm tra điều kiện dừng.
- Cập nhật $X_k = X_{k+1}$, $k = k + 1$.

B5. Trả ra nghiệm gần đúng $X^* = X_{k+1}$.

Algorithm 13 **Newton cải tiến cho hệ phi tuyến**

Input:

- Hệ $F(X) = 0$.
- Điểm đầu X_0.
- Sai số ε hoặc số vòng lặp N.
- Chuẩn chọn.

Output:

- Nghiệm gần đúng X^*.
- Ma trận $J(X_0)^{-1}$.
- Các bước lặp.

Thuật toán:

B1. Tính $J_0 = J(X_0)$. Nếu $|\det J_0| < 10^{-12}$, dừng và báo lỗi.

B2. Tính trước J_0^{-1}.

B3. Khởi tạo $k = 0$, $X_k = X_0$.

B4. Lặp:

- Tính $X_{k+1} = X_k - J_0^{-1}F(X_k)$.
- Tính sai số tuyệt đối và tương đối.
- Ghi lại bước lặp.
- Kiểm tra điều kiện dừng.
- Cập nhật $X_k = X_{k+1}$, $k = k + 1$.

B5. Trả ra nghiệm gần đúng $X^* = X_{k+1}$.

Algorithm 14 **Lặp đơn cho hệ phi tuyến**

Input:

- Ánh xạ $\Phi(X)$.
- Điểm đầu X_0.
- Tâm quả cầu α.
- Bán kính $r > 0$.
- Sai số ε hoặc số vòng lặp N.

Output:

- Nghiệm gần đúng X^*.
- Hệ số co K.
- Chuẩn được chọn.
- Bảng các bước lặp.

Thuật toán:

B1. Xét quả cầu $\overline{B}(\alpha, r) = \{X : \|X - \alpha\| \leq r\}$.

B2. Tính Jacobi $J_\Phi(X) = \dfrac{\partial \Phi}{\partial X}$.

B3. Tính các tổng hàng $R_i(X)$, tổng cột $C_j(X)$, rồi ước lượng K_∞, K_1.

B4. Đặt $K = \min(K_\infty, K_1)$. Nếu $K \geq 1$, dừng và báo không hội tụ.

B5. Chọn chuẩn tương ứng với giá trị nhỏ hơn giữa K_∞ và K_1.

B6. Lập ngưỡng dừng

$$\varepsilon_{\text{prior}} = \begin{cases} \varepsilon \dfrac{1-K}{K}, & K > 10^{-12}, \\ \varepsilon, & \text{ngược lại.} \end{cases}$$

B7. Khởi tạo $k = 0$, $X_k = X_0$.

B8. Lặp:

- Tính $X_{k+1} = \Phi(X_k)$.
- Tính $\Delta_k = X_{k+1} - X_k$.
- Tính sai số $e_k = \|\Delta_k\|_*$.
- Nếu cần, tính thêm sai số tương đối $e_k^{(\text{rel})} = \dfrac{\|\Delta_k\|_*}{\|X_{k+1}\|_*}$.
- Ghi lại bước lặp.
- Nếu $\|\Delta_k\|_* < \varepsilon_{\text{prior}}$, chuyển tới B9.

- Cập nhật $X_k = X_{k+1}$, $k = k + 1$.

B9. Trả ra nghiệm gần đúng $X^* = X_{k+1}$.